

Dagster, in plain terms

the programmable glue that runs code I write, on a schedule, against my own platform

Dagster is the lab's **data orchestrator** — the first service whose whole job is to *use* the other services. You write pipelines as Python **assets** (a thing you want to exist — a table, a report, a file), declare their dependencies, and Dagster works out the order, runs them on a schedule or trigger, records every run in Postgres, and gives you a UI to watch it all. It's in the Airflow family but **asset-first** and far nicer to look at. Mine lives on **a2** at **dagster.lan**, and the neat part is **every run executes as its own Kubernetes Job**.

Chart dagster/dagster v1.13.13

Namespace dagster (pinned to a2)

UI <https://dagster.lan/>

Argo app home-dagster

Manifests clusters/home/dagster/

3

NOUNS TO KNOW

Asset, Job, Schedule — the whole mental model

1 Job

PER RUN

K8sRunLauncher spawns a fresh pod for every run

4

DEMO ASSETS

A GPU digest that tours the whole lab in one job

6

STEP SHIP LOOP

Write → wire → validate → push → promote → verify

Dagster in one minute (the whole mental model)

You only need three nouns:

- **Asset** — a materializable output, an @asset function. *Its arguments are its dependencies*, so the DAG is inferred from the function signatures — no wiring by hand.
- **Job** — a selection of assets you run together (define_asset_job).
- **Schedule / Sensor** — *when* a job runs: a cron, or a reaction to an event.

The one idea that makes everything click

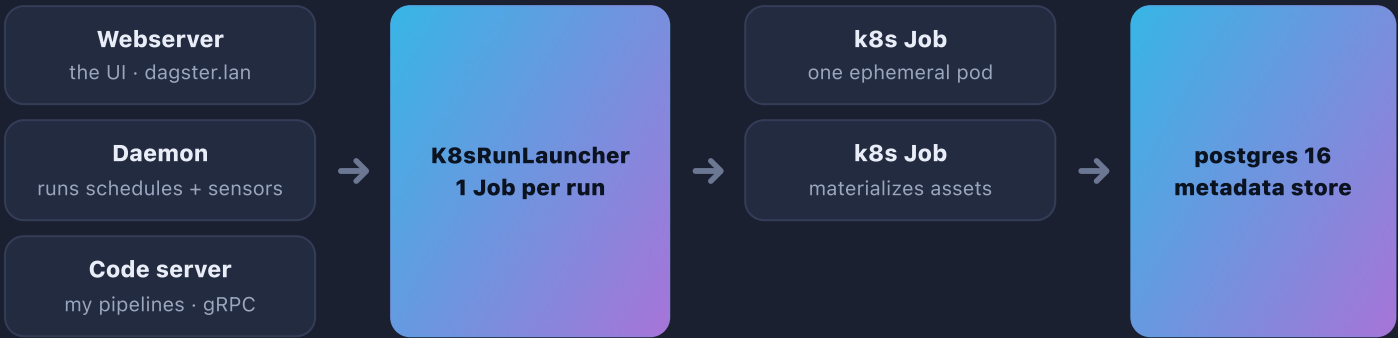
Split the world in two: **platform = the home-lab repo** (clusters/home/dagster/ — UI, daemon, Postgres, run launcher, deployed by Argo) and **pipeline code = the dagster-pipelines repo** (my assets, built into a container image and loaded over gRPC). You edit the **platform rarely** and the **pipeline code often**. Everything below follows from that split.

"An orchestrator needs a big always-on worker pool." — Unlearn this

Nothing runs inside a long-lived worker here. The **K8sRunLauncher** asks the cluster for a **fresh pod per run** and hands execution straight to the scheduler — you watch them come and go in `kubectl get jobs -n dagster`. It's a genuinely nice demonstration of k8s-native orchestration: the orchestrator delegates, the cluster executes.

How it's deployed here

The platform is the official dagster/dagster Helm chart (**v1.13.13**) inflated through `kustomize helmCharts`: — the same no-helm-CLI pattern as `open-webui` — deployed by the Argo CD app **home-dagster**. Four moving parts, plus an external Postgres.



COMPONENT	WHAT IT DOES
Webserver	The Dagster UI at <code>dagster.lan</code> — asset graph, run history, launch buttons.
Daemon	The thing that decides "it's time, go" — drives schedules and sensors.
K8sRunLauncher	Every pipeline run executes as its own Kubernetes Job. Fresh pod per run; watch them in <code>kubectyl get jobs -n dagster</code> .
User-deployments gRPC code server	A separate pod holding <i>my</i> pipeline code, serving it over gRPC so new pipelines ship without restarting Dagster itself .
Postgres	Plain <code>postgres:16-alpine</code> Deployment (repo convention — <i>not</i> the chart's bundled subchart), on a <code>local-path</code> PVC pinned to a2 .

Two sharp edges worth writing down

Postgres password ownership. The chart wants to generate its own password Secret, which would let Argo clobber the out-of-band `dagster-postgres-secret` on every sync. Set `generatePostgresqlPasswordSecret: false` — the credential is created once, out of band, and the chart is told to keep its hands off (secrets never live in git).

Why everything is pinned to a2. Run and code images live in `harbor.lan/apps/`, and pulling them requires trusting the Harbor mkcert CA. `a2` is Harbor-trusted, amd64, has the roomy e-disk, and co-locates with Postgres — the natural home. (`t430` can't host run Jobs yet only because it doesn't trust the Harbor CA — a known follow-up.)

Access & auth — mind the exposure

`dagster.lan` gets a Traefik ingress with the mkcert `dagster-tls` cert and is Homepage-discovered under the new **Data / Orchestration** group; it's also exposed remotely at `dagster.<tailnet>.ts.net` via the Tailscale operator. Critical caveat: **Dagster OSS has no built-in authentication** — no login screen. Access is gated entirely by being on the LAN or the default-deny tailnet. Acceptable for a single-operator lab; **never expose it more broadly without an auth proxy in front**.

The demonstration project: "GPU digest"

The `dagster-pipelines` repo ships one real, end-to-end pipeline plus a warm-up asset, so there's something working to learn from. It's deliberately a tour of the whole lab in one job: it reads my own **Prometheus**, thinks with my own **LiteLLM** gateway, and was shipped through my own **Forgejo + Harbor**. That's the pattern every new pipeline follows.

ASSET	WHAT IT DOES
hello_dagster	Trivial smoke-test — the first thing to materialize in the UI.
gpu_snapshot	Queries the lab's Prometheus (<code>dcgm-exporter DCGM_FI_DEV_*</code>) for per-GPU utilization, VRAM, power, temperature.
vram_by_pod	Top GPU-memory consumers by pod, from the custom vram-reporter metric.
cluster_summary	Feeds those numbers to the LiteLLM gateway and gets back a natural-language "state of the GPU fleet" digest — with a templated fallback if the gateway is down .



`gpu_snapshot` and `vram_by_pod` fan into `cluster_summary`, and the job runs on a **daily schedule** (`daily_gpu_digest`, 07:00). The model is configurable via a `LITELLM_MODEL` env (no image rebuild to switch); it defaults to the local **nemotron-omni**. So the digest is my own hardware, described by my own model, orchestrated by my own scheduler — no cloud in the loop at all.

The repo shape (worth copying)

PATH	ROLE
<code>dagster_pipelines/resources.py</code>	The seams to the cluster: <code>PrometheusResource</code> and <code>LiteLLMResource</code> . New external systems get a new resource here — never a URL hard-coded in an asset.
<code>dagster_pipelines/assets.py</code>	The assets themselves.
<code>dagster_pipelines/definitions.py</code>	Wires assets + resources + the schedule.
<code>Dockerfile</code> + <code>.forgejo/workflows/build.yaml</code>	The code-server image and its CI.

War story — the digest that named a GPU pod `DRIVERS_LICENSE_1`

The first digest described utilization for a pod called `DRIVERS_LICENSE_1`. Nothing was broken — that was **Rampart**, the PII-redaction guard, doing its job. Every prompt to LiteLLM passes through Rampart, and a hashed pod name looks enough like an ID number that the redactor swapped it for a placeholder before the model saw it. Great proof the security layer is really in the path; the obvious next refinement is normalizing pod names before they hit the gateway.

| The workflow for a new pipeline

Same GitOps machinery as every other app in the lab — push code, CI builds, Argo deploys — with one Dagster-specific twist: **promoting the image tag**. The platform lives in the home-lab repo; the pipelines live in `brian/dagster-pipelines` on Forgejo.

1 Write the module

New `dagster_pipelines/<name>.py` — the assets, plus a new resource in `resources.py` if it talks to something new.

2 Wire it into `definitions.py`

Add a job and, if it's scheduled, a schedule.

3 Validate locally

`dagster definitions validate -m dagster_pipelines.definitions` — catches import and wiring errors before CI.

4 Push → Forgejo Actions builds the image

Pushing the `dagster-pipelines` repo triggers the runner (a2, dind) to build and push `harbor.lan/apps/dagster-pipelines:<sha>`.

5 Promote — bump the tag in `values.yaml`

Set the `dagster-user-deployments` image tag in `clusters/home/dagster/values.yaml` to that `<sha>` and push home-lab. Argo rolls the code server onto the new image.

6 Verify with a K8s-Job run

The new code location loads in the UI; launch the job and watch its Kubernetes Job appear in `kubectl get jobs -n dagster`.

The version rule CI enforces for you

Pin `dagster==1.13.13`, and remember the integration libraries use the `0.X.Y` scheme that trails core by a fixed offset — so `dagster-postgres` / `dagster-k8s` are `0.29.13` for core `1.13.13`. They have to be in the image or the run pods fail to import.

Pin an explicit tag, don't chase `:latest`

Promoting by pinning the exact `<sha>` (rather than following a moving `:latest`) keeps deploys reviewable in git and trivially rollback-able — the same discipline as every other GitOps app in the lab.

The easy button: just ask Claude

Because this is my repo and Claude has the context, I don't do the six steps by hand — I describe the pipeline and let Claude run the loop. This repo carries a **Claude skill** at `.claude/skills/dagster-project/` that encodes every convention: where code goes, the resource patterns, the credentials (forgejo-bot, harbor-robot-apps-ci via `scripts/vault-secret.sh`), the a2 pinning, and the push → build → promote → verify loop.

So the prompt is just the *idea*:

```
"Make a Dagster project that pulls the top Hacker News stories, runs each
headline through LiteLLM for a one-line sentiment, and materializes a daily
table. Put it in the dagster-pipelines repo and ship it."
```

Claude will scaffold the module, wire the job and schedule, validate it, push it, watch the Harbor build, bump the tag in `values.yaml`, let Argo roll it, and launch a verification run — then report the result. Good things to include in the prompt:

SAY THIS	SO CLAUDE KNOWS...
The data source	"read Prometheus / hit this API / list this MinIO bucket" — what to build a resource for.
The output	"a daily table", "a Slack message", "a Markdown report in the UI".
The cadence	"daily at 7am", "on demand only" — whether to add a schedule.
Where it goes	Default is a new module in <code>dagster-pipelines</code> ; say so if you want a whole separate code location instead.

Starter ideas worth trying

HN → sentiment

Top Hacker News stories, one-line LLM sentiment per headline, materialized to a daily table.

MinIO bucket catalog

Counts, sizes, and newest objects per bucket — a lightweight inventory that runs on a schedule.

Backup-freshness check

Verify the nightly restic targets are recent; alert through the existing Telegram / Alertmanager stack.

GPU digest (shipped)

The flagship — Prometheus + `vram-reporter` → LiteLLM narrative, daily. The template to copy.

Conventions to keep

- A Resources for anything external**
Put URLs and clients in `resources.py`, not in `assets`. Assets stay pure and testable; the cluster seams live in one place.
- B Degrade gracefully**
Never fail a run because an *optional* dependency (the LLM, a flaky API) is down — catch it and fall back, like `cluster_summary` does.
- C Rich UI metadata**
Emit Markdown tables via `context.add_output_metadata` so a run tells its story in the UI, not just in logs.
- D Local models by default**
LLM steps default to the local `nemotron-omni` (set via `LITELLM_MODEL` — no rebuild to switch). Cloud models route through the **Rampart** PII guard, which can redact identifiers.
- E No auth — mind the exposure**
Dagster OSS has no login. Fine on the LAN and default-deny tailnet; never put it anywhere more public without an auth proxy.

Bottom line

Dagster is the lab's **programmable glue** — the layer that runs code you write, on a schedule, against the platform you already built. The platform is a handful of pods in `clusters/home/dagster/` deployed by Argo; the pipelines live in `dagster-pipelines` and ride the same Forgejo → Harbor → Argo loop as every other app, with one twist (promote the image tag). The GPU digest proves the whole loop end-to-end: Prometheus in, LiteLLM narrative out, shipped through your own forge. And for the next pipeline, you don't run the six steps by hand — you describe the idea and let the `dagster-project` skill do it.

Quick reference

THING	VALUE
UI	<code>https://dagster.lan/ · remote dagster.<tailnet>.ts.net</code>
Platform manifests	<code>clusters/home/dagster/</code> — Argo app <code>home-dagster</code>
Pipeline code	<code>brian/dagster-pipelines</code> (Forgejo) → <code>harbor.lan/apps/dagster-pipelines:<sha></code>
Promote a build	bump image tag in <code>clusters/home/dagster/values.yaml</code> , push <code>home-lab</code>
Validate	<code>dagster definitions validate -m dagster_pipelines.definitions</code>
Versions	<code>dagster==1.13.13</code> ; <code>dagster-postgres/dagster-k8s==0.29.13</code>
Watch runs	<code>kubectl get jobs -n dagster</code>
Just ask Claude	skill at <code>.claude/skills/dagster-project/</code>